

Code smells and best practices for clean code

Good coding practice is writing code that is not only effective, but also easy to read, adjust, and maintain.

Good coding practice is writing code that is not only effective, but also easy to read, adjust, and maintain.

It is more like **writing** than it is like problem-solving:

Good coding practice is writing code that is not only effective, but also easy to read, adjust, and maintain.

It is more like **writing** than it is like problem-solving:

- requires psychological insight

Good coding practice is writing code that is not only effective, but also easy to read, adjust, and maintain.

It is more like **writing** than it is like problem-solving:

- requires psychological insight
- theory of mind

Good coding practice is writing code that is not only effective, but also easy to read, adjust, and maintain.

It is more like **writing** than it is like problem-solving:

- requires psychological insight
- theory of mind
- thinking about the competence and working memory capacity of the reader

Good coding practice is writing code that is not only effective, but also easy to read, adjust, and maintain.

It is more like **writing** than it is like problem-solving:

- requires psychological insight
- theory of mind
- thinking about the competence and working memory capacity of the reader

Good coding practice is writing code that is not only effective, but also easy to read, adjust, and maintain.

It is more like **writing** than it is like problem-solving:

- requires psychological insight
- theory of mind
- thinking about the competence and working memory capacity of the reader

More often than not, good coding practice is what **not** to do.

Good coding practice is writing code that is not only effective, but also easy to read, adjust, and maintain.

It is more like **writing** than it is like problem-solving:

- requires psychological insight
- theory of mind
- thinking about the competence and working memory capacity of the reader

More often than not, good coding practice is what **not** to do.

With experience, you gain instincts that tell you when a piece of code is poorly written, hard to understand, or prone to generate bugs or runtime errors.

Good coding practice is writing code that is not only effective, but also easy to read, adjust, and maintain.

It is more like **writing** than it is like problem-solving:

- requires psychological insight
- theory of mind
- thinking about the competence and working memory capacity of the reader

More often than not, good coding practice is what **not** to do.

With experience, you gain instincts that tell you when a piece of code is poorly written, hard to understand, or prone to generate bugs or runtime errors.

And then other people's code can start to **smell bad**.

Definition

Definition

Code smells are **subjective indicators of potential problems in code** that can impact the **quality** and **maintainability** of software.

Definition

Code smells are **subjective indicators of potential problems in code** that can impact the **quality** and **maintainability** of software.

- Whether something is a code smell is an **informal judgment** about the quality of code, identifying issues such as code duplication, long method lengths, complexity, and lack of encapsulation.

Definition

Code smells are **subjective indicators of potential problems in code** that can impact the **quality** and **maintainability** of software.

- Whether something is a code smell is an **informal judgment** about the quality of code, identifying issues such as code duplication, long method lengths, complexity, and lack of encapsulation.
- Code smells are not bugs or errors, but they make code more difficult to understand, maintain, and extend, and error-prone.

Definition

Code smells are **subjective indicators of potential problems in code** that can impact the **quality** and **maintainability** of software.

- Whether something is a code smell is an **informal judgment** about the quality of code, identifying issues such as code duplication, long method lengths, complexity, and lack of encapsulation.
- Code smells are not bugs or errors, but they make code more difficult to understand, maintain, and extend, and error-prone.
- Smelly code is not broken – it works, but...

Definition

Code smells are **subjective indicators of potential problems in code** that can impact the **quality** and **maintainability** of software.

- Whether something is a code smell is an **informal judgment** about the quality of code, identifying issues such as code duplication, long method lengths, complexity, and lack of encapsulation.
- Code smells are not bugs or errors, but they make code more difficult to understand, maintain, and extend, and error-prone.
- Smelly code is not broken – it works, but...
- Code smells can lead to bugs, make code more difficult to maintain, and increase the effort required to fix problems.

Definition

Code smells are **subjective indicators of potential problems in code** that can impact the **quality** and **maintainability** of software.

- Whether something is a code smell is an **informal judgment** about the quality of code, identifying issues such as code duplication, long method lengths, complexity, and lack of encapsulation.
- Code smells are not bugs or errors, but they make code more difficult to understand, maintain, and extend, and error-prone.
- Smelly code is not broken – it works, but...
- Code smells can lead to bugs, make code more difficult to maintain, and increase the effort required to fix problems.

Definition

Code smells are **subjective indicators of potential problems in code** that can impact the **quality** and **maintainability** of software.

- Whether something is a code smell is an **informal judgment** about the quality of code, identifying issues such as code duplication, long method lengths, complexity, and lack of encapsulation.
- Code smells are not bugs or errors, but they make code more difficult to understand, maintain, and extend, and error-prone.
- Smelly code is not broken – it works, but...
- Code smells can lead to bugs, make code more difficult to maintain, and increase the effort required to fix problems.

Code smells are heuristics that indicate when to refactor.

There are many code smells that are so common they've been given names.

There are many code smells that are so common they've been given names.

The general idea is that a feeling of darkness and despair comes over you when you see some code and imagine having to do maintenance on it in the future:

There are many code smells that are so common they've been given names.

The general idea is that a feeling of darkness and despair comes over you when you see some code and imagine having to do maintenance on it in the future:

- This is hard to read.

There are many code smells that are so common they've been given names.

The general idea is that a feeling of darkness and despair comes over you when you see some code and imagine having to do maintenance on it in the future:

- This is hard to read.
- This is hard to test.

There are many code smells that are so common they've been given names.

The general idea is that a feeling of darkness and despair comes over you when you see some code and imagine having to do maintenance on it in the future:

- This is hard to read.
- This is hard to test.
- If I change this, things elsewhere break.

There are many code smells that are so common they've been given names.

The general idea is that a feeling of darkness and despair comes over you when you see some code and imagine having to do maintenance on it in the future:

- This is hard to read.
- This is hard to test.
- If I change this, things elsewhere break.
- I find myself changing this back and forth.



Mysterious name

- Functions, modules, variables or classes that are named in a way that does not communicate what they do or how to use them.



Mysterious name

- Functions, modules, variables or classes that are named in a way that does not communicate what they do or how to use them.



Mysterious name

- Functions, modules, variables or classes that are named in a way that does not communicate what they do or how to use them.

`doTheThing(a,aa,flag)`



Mysterious name

- Functions, modules, variables or classes that are named in a way that does not communicate what they do or how to use them.

`doTheThing(a,aa,flag)`

- Use meaningful and descriptive names.



Mysterious name

- Functions, modules, variables or classes that are named in a way that does not communicate what they do or how to use them.

`doTheThing(a,aa,flag)`

- Use meaningful and descriptive names.



Mysterious name

- Functions, modules, variables or classes that are named in a way that does not communicate what they do or how to use them.

```
doTheThing(a,aa,flag)
```

- Use meaningful and descriptive names.

```
marginalLikelihood(binomialData, prior, useCache)
```

- Functions, modules, variables or classes that are named in a way that does not communicate what they do or how to use them.

`doTheThing(a,aa,flag)`

- Use meaningful and descriptive names.

`marginalLikelihood(binomialData, prior, useCache)`

- Use naming conventions and abbreviations that are widely recognized and understood.

- Functions, modules, variables or classes that are named in a way that does not communicate what they do or how to use them.

`doTheThing(a,aa,flag)`

- Use meaningful and descriptive names.

`marginalLikelihood(binomialData, prior, useCache)`

- Use naming conventions and abbreviations that are widely recognized and understood.

Mysterious name

- Functions, modules, variables or classes that are named in a way that does not communicate what they do or how to use them.

```
doTheThing(a,aa,flag)
```

- Use meaningful and descriptive names.

```
marginalLikelihood(binomialData, prior, useCache)
```

- Use naming conventions and abbreviations that are widely recognized and understood.

```
margLik(binomData, prior, useCache)
```



Duplicated code

- Identical or very similar code exists in more than one location.



Duplicated code

- Identical or very similar code exists in more than one location.



Duplicated code

- Identical or very similar code exists in more than one location.

```
np.mean(samples[0]['alpha'].flatten())  
np.mean(samples[0]['gamma'].flatten())  
np.mean(samples[0]['delta'].flatten())
```



Duplicated code

- Identical or very similar code exists in more than one location.

```
np.mean(samples[0]['alpha'].flatten())
```

```
np.mean(samples[0]['gamma'].flatten())
```

```
np.mean(samples[0]['delta'].flatten())
```

- Extract duplicated code into a reusable function or class.



Duplicated code

- Identical or very similar code exists in more than one location.

```
np.mean(samples[0]['alpha'].flatten())
```

```
np.mean(samples[0]['gamma'].flatten())
```

```
np.mean(samples[0]['delta'].flatten())
```

- Extract duplicated code into a reusable function or class.



Duplicated code

- Identical or very similar code exists in more than one location.

```
np.mean(samples[0]['alpha'].flatten())  
np.mean(samples[0]['gamma'].flatten())  
np.mean(samples[0]['delta'].flatten())
```

- Extract duplicated code into a reusable function or class.

```
def chainMean(samples, parameter):  
    chain = samples[0][parameter].flatten()  
    return numpy.mean(chain)
```

Duplicated code — calls and tests

```
chainMean(samples, 'alpha')  
chainMean(samples, 'gamma')  
chainMean(samples, 'delta')
```



```
chainMean(samples, 'alpha')  
chainMean(samples, 'gamma')  
chainMean(samples, 'delta')
```

- Now you can ensure that the extracted code is tested, so that it can be easily checked, maintained, and updated.

```
chainMean(samples, 'alpha')  
chainMean(samples, 'gamma')  
chainMean(samples, 'delta')
```

- Now you can ensure that the extracted code is tested, so that it can be easily checked, maintained, and updated.

```
chainMean(samples, 'alpha')  
chainMean(samples, 'gamma')  
chainMean(samples, 'delta')
```

- Now you can ensure that the extracted code is tested, so that it can be easily checked, maintained, and updated.

```
def testChainMean(unittest.TestCase):  
    ...
```



- Single changes often need to be applied to multiple classes or methods at the same time.



- Single changes often need to be applied to multiple classes or methods at the same time.



- Single changes often need to be applied to multiple classes or methods at the same time.

```
def integrand1(self, p):  
    return self.pdf(p, self.n, self.k) * self.prior1(p)  
def integrand2(self, p):  
    return self.pdf(p, self.n, self.k) * self.prior2(p)
```



- Identify the commonality between the classes that need to be modified and extract it into a common class or function.



- Identify the commonality between the classes that need to be modified and extract it into a common class or function.



- Identify the commonality between the classes that need to be modified and extract it into a common class or function.

```
def integrand(self, prior, p):  
    return self.pdf(p, self.n, self.k) * prior(p)
```

- Identify the commonality between the classes that need to be modified and extract it into a common class or function.

```
def integrand(self, prior, p):  
    return self.pdf(p, self.n, self.k) * prior(p)
```

- Manage responsibilities



- Identify the commonality between the classes that need to be modified and extract it into a common class or function.

```
def integrand(self, prior, p):  
    return self.pdf(p, self.n, self.k) * prior(p)
```

- Manage responsibilities
 - A class or method should generally have **one** responsibility.



- Identify the commonality between the classes that need to be modified and extract it into a common class or function.

```
def integrand(self, prior, p):  
    return self.pdf(p, self.n, self.k) * prior(p)
```

- Manage responsibilities
 - A class or method should generally have **one** responsibility.
 - Don't split a single responsibility among classes or methods.

- Frequently changing what a particular identifier refers to.

- Frequently changing what a particular identifier refers to.



Variable mutations — avoid rebinding

- Frequently changing what a particular identifier refers to.

```
from math import comb
```

```
def binomial_pdf(obj, p):  
    p = comb(obj.n, obj.k) * (p**obj.k) * ((1 - p)**(obj.n - obj.k))  
    return p
```

- Avoid changing the reference of a variable, prefer creating new variables.

Variable mutations — clearer names

- Avoid changing the reference of a variable, prefer creating new variables.
- Consider using constants or read-only variables where appropriate to reduce the likelihood of unexpected mutations.

Variable mutations — clearer names

- Avoid changing the reference of a variable, prefer creating new variables.
- Consider using constants or read-only variables where appropriate to reduce the likelihood of unexpected mutations.

- Avoid changing the reference of a variable, prefer creating new variables.
- Consider using constants or read-only variables where appropriate to reduce the likelihood of unexpected mutations.

```
from math import comb
```

```
def binomial_pdf(obj, p):  
    n_win, n_lose = obj.get_k(), obj.get_n() - obj.get_k()  
    p_win, p_lose = p, 1 - p  
    out = comb(n_win + n_lose, n_win) * (p_win**n_win) * (p_lose**n_lose)  
    return out
```

Uncontrolled side effects

- Methods modify variables beyond scope (like globals or complex variables passed by reference).

Uncontrolled side effects

- Methods modify variables beyond scope (like globals or complex variables passed by reference).
- Refactor the code to avoid modifying variables outside of the method scope.

Uncontrolled side effects

- Methods modify variables beyond scope (like globals or complex variables passed by reference).
- Refactor the code to avoid modifying variables outside of the method scope.
- Always use functional programming techniques, such as passing data through function arguments and return values, to reduce the likelihood of uncontrolled side effects.

Uncontrolled side effects

- Methods modify variables beyond scope (like globals or complex variables passed by reference).
- Refactor the code to avoid modifying variables outside of the method scope.
- Always use functional programming techniques, such as passing data through function arguments and return values, to reduce the likelihood of uncontrolled side effects.

Uncontrolled side effects

- Methods modify variables beyond scope (like globals or complex variables passed by reference).
- Refactor the code to avoid modifying variables outside of the method scope.
- Always use functional programming techniques, such as passing data through function arguments and return values, to reduce the likelihood of uncontrolled side effects.

Respect scope.


```
class myClass:  
    def __init__(self, X):  
        self.X = X
```

```
def getX(obj):  
    X = obj.X  
    obj.X = None  
    return X
```

```
n = myClass(5)
```

```
(n.X, getX(n), n.X)
```

APIs should **read** state without surprising the caller by clearing fields as a side effect. Compare:

APIs should **read** state without surprising the caller by clearing fields as a side effect. Compare:

```
class Wallet:
    def __init__(self, balance):
        self.balance = balance

def peek_balance(wallet: Wallet) -> int:
    return wallet.balance

w = Wallet(5)
(w.balance, peek_balance(w), w.balance)  # --> (5, 5, 5)
```



Data clumps

- Multiple related variables that are often passed together, indicating a need for a class.



Data clumps

- Multiple related variables that are often passed together, indicating a need for a class.
- Refactor the code to extract the related variables into a class.



Data clumps

- Multiple related variables that are often passed together, indicating a need for a class.
- Refactor the code to extract the related variables into a class.
- Easier to understand and maintain.



Data clumps

- Multiple related variables that are often passed together, indicating a need for a class.
- Refactor the code to extract the related variables into a class.
- Easier to understand and maintain.



Data clumps

- Multiple related variables that are often passed together, indicating a need for a class.
- Refactor the code to extract the related variables into a class.
- Easier to understand and maintain.

```
def show_fixation(width_px, height_px, distance_cm, gamma, bg_color):  
    ...
```

```
def run_trial(subject_id, width_px, height_px, distance_cm, gamma, bg_color):  
    show_fixation(width_px, height_px, distance_cm, gamma, bg_color)  
    # Same display settings travel together on every call: a clump.
```




Data clumps

- Multiple related variables that are often passed together, indicating a need for a class.
- Refactor the code to extract the related variables into a class.
- Easier to understand and maintain.

```
def show_fixation(width_px, height_px, distance_cm, gamma, bg_color):  
    ...
```

```
def run_trial(subject_id, width_px, height_px, distance_cm, gamma, bg_color):  
    show_fixation(width_px, height_px, distance_cm, gamma, bg_color)  
    # Same display settings travel together on every call: a clump.
```

```
from dataclasses import dataclass
```

- Writing code for functionality that may not be needed in the future adds unnecessary complexity and makes the code difficult to maintain.

- Writing code for functionality that may not be needed in the future adds unnecessary complexity and makes the code difficult to maintain.

- Writing code for functionality that may not be needed in the future adds unnecessary complexity and makes the code difficult to maintain.

```
def mean(xs, weights=None, ignore_nans=False, trim_fraction=0.0):  
    return sum(xs) / len(xs)
```

- Every possible path through a function adds complexity.

- Every possible path through a function adds complexity.
- Many independent branches means many paths to read, test, and change safely.

- Every possible path through a function adds complexity.
- Many independent branches means many paths to read, test, and change safely.
- It may be possible to simplify the logic, or this needs to be multiple functions.

- Every possible path through a function adds complexity.
- Many independent branches means many paths to read, test, and change safely.
- It may be possible to simplify the logic, or this needs to be multiple functions.

- Every possible path through a function adds complexity.
- Many independent branches means many paths to read, test, and change safely.
- It may be possible to simplify the logic, or this needs to be multiple functions.

```
def label_trial(rt_ms, correct, practice):  
    if practice:  
        if rt_ms < 150:  
            if not correct:  
                return "practice_fast_error"  
            return "practice_fast_correct"  
        if rt_ms > 3000:  
            if not correct:  
                return "practice_slow_error"  
            return "practice_slow_correct"
```

Too many parameters — crowded signature

- Makes calling and testing a function complicated

Too many parameters — crowded signature

- Makes calling and testing a function complicated

Too many parameters — crowded signature

- Makes calling and testing a function complicated

```
def plot_experiment(  
    iv_data,  
    iv_names,  
    dv_data,  
    dv_names,  
    covariates,  
    covariate_names,  
    experiment_name,  
    aesthetic,  
):  
    ... # do stuff ...
```

Too many parameters — reshape responsibility

- May indicate that the purpose of the function is ill-conceived

Too many parameters — reshape responsibility

- May indicate that the purpose of the function is ill-conceived
- Refactor so responsibility is assigned in a more clean-cut way

Too many parameters — reshape responsibility

- May indicate that the purpose of the function is ill-conceived
- Refactor so responsibility is assigned in a more clean-cut way

Too many parameters — reshape responsibility

- May indicate that the purpose of the function is ill-conceived
- Refactor so responsibility is assigned in a more clean-cut way

```
class Experiment:
```

```
...    # Class to contain experiment data ...
```


- A method that has grown too large

Long method

- A method that has grown too large
- Difficult to understand and maintain



Long method

- A method that has grown too large
- Difficult to understand and maintain
- Refactor into smaller, more focused methods that each perform a single, specific task

Long method

- A method that has grown too large
- Difficult to understand and maintain
- Refactor into smaller, more focused methods that each perform a single, specific task

Long method

- A method that has grown too large
- Difficult to understand and maintain
- Refactor into smaller, more focused methods that each perform a single, specific task

```
run_simulation_study(n_repetitions):  
    parameters          = sample_parameters(n_repetitions)  
    simulated_data       = simulate_data(parameters)  
    estimated_parameters = estimate_parameters(simulated_data)  
    return compare_parameters(  
        true_parameters      = parameters,  
        estimated_parameters = estimated_parameters  
    )
```



Excessively long or short identifiers

- Naming conventions used to provide disambiguation that should be implicit in the software architecture



Excessively long or short identifiers

- Naming conventions used to provide disambiguation that should be implicit in the software architecture
- Can make code harder to read and understand



Excessively long or short identifiers

- Naming conventions used to provide disambiguation that should be implicit in the software architecture
- Can make code harder to read and understand



Excessively long or short identifiers

- Naming conventions used to provide disambiguation that should be implicit in the software architecture
- Can make code harder to read and understand

```
compute_marginal_likelihood_from_binomial_data_and_priors(  
    binomialData, prior, useCache  
)
```



Excessively long or short identifiers

- Naming conventions used to provide disambiguation that should be implicit in the software architecture
- Can make code harder to read and understand

```
compute_marginal_likelihood_from_binomial_data_and_priors(  
    binomialData, prior, useCache  
)  
  
ml(d1, pr, uc)
```



Excessively long or short identifiers

- Use concise but descriptive names



Excessively long or short identifiers

- Use concise but descriptive names



Excessively long or short identifiers

- Use concise but descriptive names

```
marginalLikelihood(binomialData, prior, useCache)
```

Excessively long line of code — dense expression

- Makes code difficult to read, understand, debug, refactor, or identify possibilities for software reuse

Excessively long line of code — dense expression

- Makes code difficult to read, understand, debug, refactor, or identify possibilities for software reuse

Excessively long line of code — dense expression

- Makes code difficult to read, understand, debug, refactor, or identify possibilities for software reuse

```
subtab = data.loc[subset].groupby(  
    factors  
)[value].agg(  
    [numpy.mean, numpy.std, len]  
)
```


Excessively long line of code — stepped layout

- Break up long lines into smaller, more manageable chunks

Excessively long line of code — stepped layout

- Break up long lines into smaller, more manageable chunks

Excessively long line of code — stepped layout

- Break up long lines into smaller, more manageable chunks

```
statistics_list = [numpy.mean, numpy.std, len]
subset         = data.loc[subset]
grouped_subset = subset.groupby(factors)
grouped_value  = grouped_subset[value]
aggregated_value = grouped_value.agg(statistics_list)
```

- Excessive comments betray you have something to apologize for

- Excessive comments betray you have something to apologize for
- A comment on an attribute setter/getter is a good example



- Excessive comments betray you have something to apologize for
- A comment on an attribute setter/getter is a good example
- Can make code harder to read and maintain because a change in code leads to a change in comment



- Excessive comments betray you have something to apologize for
- A comment on an attribute setter/getter is a good example
- Can make code harder to read and maintain because a change in code leads to a change in comment
- Write short methods with descriptive titles so they don't need much explanation



- Excessive comments betray you have something to apologize for
- A comment on an attribute setter/getter is a good example
- Can make code harder to read and maintain because a change in code leads to a change in comment
- Write short methods with descriptive titles so they don't need much explanation
- Practice **contractual programming**: document the **interface** carefully



-  Large class: a class that contains many unrelated methods.



Class-level smells

-  Large class: a class that contains many unrelated methods.
-  Lazy class: a class that does too little.



- Large class: a class that contains many unrelated methods.
- Lazy class: a class that does too little.
- Feature envy: a class that uses methods of another class excessively.



- **Large class:** a class that contains many unrelated methods.
- **Lazy class:** a class that does too little.
- **Feature envy:** a class that uses methods of another class excessively.
- **Excessive use of literals:** these should be coded as named constants, to improve readability and to avoid programming errors.

- Use clear and descriptive names for variables, functions, classes, and other identifiers.

- Use clear and descriptive names for variables, functions, classes, and other identifiers.
- The names should clearly indicate the purpose and behavior of the code.

- Use clear and descriptive names for variables, functions, classes, and other identifiers.
- The names should clearly indicate the purpose and behavior of the code.
- Example:

- Use clear and descriptive names for variables, functions, classes, and other identifiers.
- The names should clearly indicate the purpose and behavior of the code.
- Example:
 - Instead of using a variable name like `x`, use `customerName`

- Use clear and descriptive names for variables, functions, classes, and other identifiers.
- The names should clearly indicate the purpose and behavior of the code.
- Example:
 - Instead of using a variable name like `x`, use `customerName`
 - Instead of using a function name like `process`, use `calculateDiscount`



Naming conventions — bad practice

- Bad naming practices can make code difficult to read and understand



Naming conventions — bad practice

- Bad naming practices can make code difficult to read and understand

Naming conventions — bad practice

- Bad naming practices can make code difficult to read and understand

Bad naming practice

```
lst = [1, 2, 3]
```

```
dct = {"a": 1, "b": 2}
```

```
def fnc(a, b):  
    return a + b
```

```
class Cls:  
    def __init__(self):  
        self.x = None  
        self.y = None  
    def mtd(self):  
        pass
```

Naming conventions — good practice

- Good naming practices make code more self-documenting and easier to maintain

Naming conventions — good practice

- Good naming practices make code more self-documenting and easier to maintain

Naming conventions — good practice

- Good naming practices make code more self-documenting and easier to maintain

Good naming practice

```
numbers_list = [1, 2, 3]
```

```
data_dict = {"height": 1, "weight": 2}
```

```
def addNumbers(firstNumber, secondNumber):  
    return firstNumber + secondNumber
```

```
class DataProcessor:  
    def __init__(self):  
        self.current_data = None  
        self.previous_data = None  
    def process_data(self):  
        pass
```

- Keep functions small and focused.

- Keep functions small and focused.
- A function should do one thing and do it well.

- Keep functions small and focused.
- A function should do one thing and do it well.
- Avoid long functions with multiple responsibilities.

- Keep functions small and focused.
- A function should do one thing and do it well.
- Avoid long functions with multiple responsibilities.
- Example:

- Keep functions small and focused.
- A function should do one thing and do it well.
- Avoid long functions with multiple responsibilities.
- Example:
 - Instead of having a long function that retrieves data from a database, processes it, and sends an email all in one, have three separate functions: one for retrieving data, one for processing data, and one for sending emails.



```
def calculate_and_save_results(numbers):  
    result = 0  
    for number in numbers:  
        result += number  
    with open("result.txt", "w") as f:  
        f.write(str(result))  
    return result
```



Refactor functions — after

```
def calculate_sum(numbers):  
    result = 0  
    for number in numbers:  
        result += number  
    return result
```

```
def save_to_file(result, file_path):  
    with open(file_path, "w") as f:  
        f.write(str(result))
```

```
numbers = [1, 2, 3, 4]  
result = calculate_sum(numbers)  
save_to_file(result, "result.txt")
```

- Use comments only when they provide additional information that is not already clear from the code.



- Use comments only when they provide additional information that is not already clear from the code.
- Avoid writing comments that simply repeat the code.



- Use comments only when they provide additional information that is not already clear from the code.
- Avoid writing comments that simply repeat the code.
- Example:



- Use comments only when they provide additional information that is not already clear from the code.
- Avoid writing comments that simply repeat the code.
- Example:
 - Instead of writing a comment like “increment the value of x” before “x += 1”... don’t write that



- Use comments only when they provide additional information that is not already clear from the code.
- Avoid writing comments that simply repeat the code.
- Example:
 - Instead of writing a comment like “increment the value of x” before “x += 1”... don’t write that
- If you find you need to explain your code a lot, you might want to simplify the code rather than apologizing for it



Bad commenting practice

```
def calculate_sum(numbers):  
    # Calculating the sum of numbers  
    result = 0  
    for number in numbers:  
        result += number  
    # Return result  
    return result
```



```
def calculate_sum(numbers):  
    """  
    Calculate the sum of a list of numbers.  
  
    Arguments:  
    numbers -- list of numbers to be summed  
  
    Returns:  
    result -- the sum of the numbers  
    """  
    result = 0  
    for number in numbers:  
        result += number  
    return result
```



- Use consistent formatting throughout the codebase.



Formatting

- Use consistent formatting throughout the codebase.
- This makes the code easier to read and understand.



- Use consistent formatting throughout the codebase.
- This makes the code easier to read and understand.
- Example:



- Use consistent formatting throughout the codebase.
- This makes the code easier to read and understand.
- Example:
 - Use a consistent indentation level throughout the code.



- Use consistent formatting throughout the codebase.
- This makes the code easier to read and understand.
- Example:
 - Use a consistent indentation level throughout the code.
 - Use consistent capitalization for variables and functions.



- Use consistent formatting throughout the codebase.
- This makes the code easier to read and understand.
- Example:
 - Use a consistent indentation level throughout the code.
 - Use consistent capitalization for variables and functions.
 - Use consistent spacing and alignment.

```
def calculate_sum(numbers):  
    output_arg=0  
    for i in range(len(numbers)): output_arg=output_arg+numbers[i]  
    return output_arg
```

```
def calculate_sum(numbers):  
    output_arg = 0  
    for i in range(len(numbers)):  
        output_arg = output_arg + numbers[i]  
    return output_arg
```



Error handling

- Handle errors/exceptions in a consistent and predictable way.



Error handling

- Handle errors/exceptions in a consistent and predictable way.
- Use exceptions to indicate **what** has gone wrong.



- Handle errors/exceptions in a consistent and predictable way.
- Use exceptions to indicate **what** has gone wrong.
- In general, do not try to fix a user's input error. Wrong input should give an informative error message to set the user straight. **Code should never quietly give a result that might not be what the user wanted.**

Good Error Handling Practice

✓ Error handling — good practice

Good Error Handling Practice

```
def centered_normpdf(x, sigma):  
    if sigma == 0:  
        raise ValueError("The standard deviation" +  
                           " must be non-zero.")  
    return (1 / (math.sqrt(2 * math.pi) * sigma)) * \  
           math.exp(-0.5 * (x / sigma) ** 2)  
  
print(centered_normpdf(0, 0))  
# ValueError: The standard deviation must be non-zero.
```

Inferior Error Handling Practice



Inferior Error Handling Practice

```
def centered_normpdf(x, s):  
    return (1 / (math.sqrt(2 * math.pi) * s)) * \  
        math.exp(-0.5 * (x / s) ** 2)  
  
print(centered_normpdf(0, 0))  
# ZeroDivisionError: float division by zero
```

Overly Defensive Error Handling Practice

Overly Defensive Error Handling Practice

```
def centered_normpdf(x, s):  
    s = max(0.0001, s)  
    return (1 / (math.sqrt(2 * math.pi) * s)) * \  
        math.exp(-0.5 * (x / s) ** 2)  
  
print(centered_normpdf(0, 0))  
# 3989.4228040143275
```

- Regularly review and refactor the code to improve its design and maintainability.

- Regularly review and refactor the code to improve its design and maintainability.
- Make small, incremental changes to the code rather than large, sweeping changes.

- Regularly review and refactor the code to improve its design and maintainability.
- Make small, incremental changes to the code rather than large, sweeping changes.
- Example:

- Regularly review and refactor the code to improve its design and maintainability.
- Make small, incremental changes to the code rather than large, sweeping changes.
- Example:
 - Instead of rewriting an entire module, extract the reusable parts into a separate function or class.

- Regularly review and refactor the code to improve its design and maintainability.
- Make small, incremental changes to the code rather than large, sweeping changes.
- Example:
 - Instead of rewriting an entire module, extract the reusable parts into a separate function or class.
 - Instead of adding new features to a class with many responsibilities, extract the new feature into a separate class.

- Regularly review and refactor the code to improve its design and maintainability.
- Make small, incremental changes to the code rather than large, sweeping changes.
- Example:
 - Instead of rewriting an entire module, extract the reusable parts into a separate function or class.
 - Instead of adding new features to a class with many responsibilities, extract the new feature into a separate class.
- Constant unit testing

- Regularly review and refactor the code to improve its design and maintainability.
- Make small, incremental changes to the code rather than large, sweeping changes.
- Example:
 - Instead of rewriting an entire module, extract the reusable parts into a separate function or class.
 - Instead of adding new features to a class with many responsibilities, extract the new feature into a separate class.
- Constant unit testing
 - Write unit tests to ensure that the code behaves as expected and to catch regressions.

- Regularly review and refactor the code to improve its design and maintainability.
- Make small, incremental changes to the code rather than large, sweeping changes.
- Example:
 - Instead of rewriting an entire module, extract the reusable parts into a separate function or class.
 - Instead of adding new features to a class with many responsibilities, extract the new feature into a separate class.
- Constant unit testing
 - Write unit tests to ensure that the code behaves as expected and to catch regressions.
 - A good test suite gives you confidence to make changes to the code without fear of breaking existing functionality.

- Continual learning

- Continual learning
 - Stay up-to-date with new technologies and industry best practices.

- Continual learning
 - Stay up-to-date with new technologies and industry best practices.
 - Continuously improve your skills and knowledge.

- Continual learning
 - Stay up-to-date with new technologies and industry best practices.
 - Continuously improve your skills and knowledge.
 - Example:

- Continual learning
 - Stay up-to-date with new technologies and industry best practices.
 - Continuously improve your skills and knowledge.
 - Example:
 - Read books and articles on software development.

- Continual learning
 - Stay up-to-date with new technologies and industry best practices.
 - Continuously improve your skills and knowledge.
 - Example:
 - Read books and articles on software development.
 - Attend conferences, meetups, and other events.

- Continual learning
 - Stay up-to-date with new technologies and industry best practices.
 - Continuously improve your skills and knowledge.
 - Example:
 - Read books and articles on software development.
 - Attend conferences, meetups, and other events.
 - Experiment with new technologies and programming languages.